

A Comparative Benchmark of Sorting Algorithms Across Data Structures and Input Distributions

Presentation Review Report

Presenter: Gabriel Gîrjaliu

Reviewer: Maria Maiorescu

Faculty of Computer Science
West University of Timișoara

1 Main Results Presented

The presentation compared nine sorting algorithms across different input distributions and data structures. The study examined arrays and linked lists, as well as random, sorted, reverse, almost-sorted, flat, float, and string datasets.

The results showed that quadratic algorithms become impractical for large input sizes, while Heap Sort, Merge Sort, Quick Sort, Counting Sort, and Radix Sort scale much better. One of the main conclusions was that Heap Sort achieved the best overall performance among the general-purpose comparison-based algorithms tested. The presentation also showed that input distribution can significantly affect performance, especially for Insertion Sort and Counting Sort.

2 What Worked Well in the Presentation

- The experiment considered many factors, including data structures, input distributions, and parallel execution.
- The methodology was clearly explained and easy to follow.
- The results were supported by detailed measurements and comparisons.

- The presenter highlighted both theoretical and practical aspects of sorting algorithms.
- The future work section showed awareness of the limitations of the current implementation.

3 What Did Not Work Well in the Presentation

- The amount of experimental data was quite large and sometimes difficult to absorb during a short presentation.
- Some of the performance differences could have been illustrated with additional visual graphs.
- Since the implementation was written in Python, some results may have been influenced by interpreter overhead rather than only by the algorithms themselves.

4 What I Learned from the Presentation

I learned that the choice of input distribution can dramatically change algorithm performance. I also learned that linked-list overhead was relatively small in this experiment and that Heap Sort performed better than expected in many of the tested scenarios. Additionally, I gained a better understanding of how implementation details and programming languages can influence benchmarking results.

5 Questions Asked and Answers Received

Question: Why will rewriting this experiment in C++ help you get better results?

Answer: The presenter explained that C++ runs directly on the computer hardware without Python's interpreter layer. This allows the use of low-level profiling tools that can measure CPU cache behavior and memory access patterns more accurately. As a result, it becomes easier to determine whether Heap Sort is genuinely faster or whether some of the observed performance differences were influenced by Python's runtime environment.